Algorithm 2: Population-Weighted Poll-to-Census-Tract Interpolation

Note on the population denominator

This algorithm uses population as the weighting variable, but the same method can be used with a better eligibility proxy.

For example, instead of using total population, we can substitute:

```
citizens_above_18
```

or another preferred denominator such as:

```
voting_age_population
adult_population
eligible_voters
registered_electors
```

The algorithm stays the same. The only change is the variable used to estimate where the relevant voting population lives.

In the steps below, this variable is called:

```
ancillary_weight_u
```

This can mean total population, voting-age population, citizens above 18, or another population proxy.

Goal

We want to interpolate election results from election polls onto census tracts.

The source geography is:

```
election polls
```

The target geography is:

```
census tracts
```

Each election poll has observed vote totals, including total votes and votes by party.

Each census tract is the geography we want to estimate results for.

The goal is to estimate, for each census tract:

```
estimated_total_votes_t
estimated_votes_party_A_t
estimated_votes_party_B_t
estimated_votes_party_C_t
...
estimated_party_share_A_t
estimated_party_share_B_t
estimated_party_share_C_t
...
```

Intuition

The basic idea is:

```
Do not allocate votes based only on land area.
Allocate votes based on where people likely live.
```

A purely spatial method would say:

```
If 40% of a poll's area overlaps a census tract, assign 40% of that poll's votes
to the tract.
```

The population-weighted method instead says:

```
If 40% of a poll's relevant population overlaps a census tract, assign 40% of
that poll's votes to the tract.
```

This is better because people are not evenly distributed across land area. Some parts of a poll may contain parks, industrial land, ravines, highways, commercial areas, or other places where few voters live.

This method preserves the official poll-level vote totals while producing approximate census-tract-level estimates.

Inputs

1. Election poll layer

Each election poll is indexed by `p`.

Required fields:

```
poll_id
geometry_p
total_votes_p
votes_party_A_p
votes_party_B_p
votes_party_C_p
...
```

Example:

```
poll_id
geometry
total_votes
votes_liberal
votes_conservative
votes_ndp
votes_green
votes_other
```

2. Census tract layer

Each census tract is indexed by `t`.

Required fields:

```
ct_id
geometry_t
```

Optional fields:

```
population_t
voting_age_population_t
citizens_above_18_t
```

These are useful for checking results or calculating approximate turnout, but they are not strictly required for the interpolation if a smaller ancillary population layer is available.

3. Ancillary population layer

Each ancillary unit is indexed by `u`.

This should ideally be a geography smaller than both polls and census tracts.

Examples:

```
dissemination blocks
dissemination areas
small census geographies
population grid cells
residential parcels
building footprints with population or dwelling counts
```

Required fields:

```
unit_id
geometry_u
ancillary_weight_u
```

Where:

```
ancillary_weight_u
```

is the population-like variable used for weighting.

Preferred hierarchy:

```
citizens_above_18_u
voting_age_population_u
adult_population_u
```

```
total_population_u
dwelling_count_u
```

Output

The final output is one row per census tract.

Required output fields:

```
ct_id
estimated_total_votes_t
estimated_votes_party_A_t
estimated_votes_party_B_t
estimated_votes_party_C_t
...
estimated_party_share_A_t
estimated_party_share_B_t
estimated_party_share_C_t
...
```

Optional output fields:

```
num_source_polls_t
share_from_largest_poll_t
method
quality_flags
```

Step 1: Prepare all geometries

Reproject all spatial layers to the same local projected coordinate reference system.

For Toronto, use a metre-based projection rather than latitude/longitude.

Inputs to reproject:

```
polls
census_tracts
ancillary_population_units
```

Then repair invalid geometries if needed.

The goal is to make sure all layers can be intersected accurately.

Step 2: Create poll-census-tract intersections

Create an intersection between the election polls and census tracts.

For every poll `p` and census tract `t` that overlap, create one row:

```
poll_id
ct_id
geometry_pt
```

where:

```
geometry_pt = geometry_p ∩ geometry_t
```

This creates the basic crosswalk between polls and census tracts.

Call this layer:

```
poll_ct
```

Step 3: Intersect the ancillary population layer with the poll-tract intersections

Now intersect the ancillary population units with the `poll_ct` layer.

For every ancillary unit `u`, poll `p`, and census tract `t` that overlap, create one row:

```
unit_id
poll_id
ct_id
geometry_upt
```

where:

```
geometry_upt = geometry_u ∩ geometry_p ∩ geometry_t
```

This gives the part of each small population unit that lies inside a specific poll-tract overlap.

Call this layer:

```
ancillary_poll_ct
```

Step 4: Estimate ancillary population inside each poll-tract overlap

For each row in `ancillary_poll_ct`, calculate:

```
area_u = area of the original ancillary unit u
area_upt = area of the piece geometry_upt
```

Then estimate how much of the ancillary population belongs to that piece:

```
allocated_weight_upt = ancillary_weight_u * (area_upt / area_u)
```

This assumes that the population of the ancillary unit is evenly distributed inside the ancillary unit.

Because the ancillary units are smaller than polls and census tracts, this is usually a reasonable approximation.

Step 5: Sum ancillary weights to each poll-tract pair

For each poll `p` and census tract `t`, sum the allocated weights from all ancillary units.

```
pop_weight_pt = sum(allocated_weight_upt)
```

Grouped by:

```
poll_id
ct_id
```

The result is one row per poll-tract overlap:

```
poll_id
ct_id
pop_weight_pt
```

This value estimates how much relevant population lies in the overlap between poll `p` and census tract `t`.

Step 6: Normalize weights within each poll

For each poll `p`, calculate the total population weight assigned to all census tracts it overlaps:

```
total_pop_weight_p = sum(pop_weight_pt for all t overlapping p)
```

Then calculate the allocation weight for each poll-tract overlap:

```
allocation_weight_pt = pop_weight_pt / total_pop_weight_p
```

This means that, for each poll:

```
sum(allocation_weight_pt for all t overlapping p) = 1
```

This step is important because it preserves the official vote totals from each poll.

Step 7: Handle zero-population edge cases

Sometimes a poll may have votes but no estimated population weight.

This can happen because of geometry mismatch, missing data, or small inconsistencies between datasets.

If:

```
total_votes_p > 0
```

but:

```
total_pop_weight_p = 0
```

then use a fallback method.

Recommended fallback:

```
Use area weighting for that poll.
```

For that poll only, calculate:

```
allocation_weight_pt = area(geometry_p ∩ geometry_t) / area(geometry_p)
```

Also flag that poll or its resulting census tract estimates with:

```
fallback_area_weight_used = true
```

Step 8: Allocate total votes from polls to census tracts

For each poll-tract overlap, allocate total votes using the population-based allocation weight.

```
allocated_total_votes_pt = total_votes_p * allocation_weight_pt
```

This estimates how many votes from poll `p` should be assigned to census tract `t`.

Step 9: Allocate party votes from polls to census tracts

For each party, allocate votes using the same allocation weight.

For party A:

```
allocated_votes_party_A_pt = votes_party_A_p * allocation_weight_pt
```

For party B:

```
allocated_votes_party_B_pt = votes_party_B_p * allocation_weight_pt
```

For party C:

```
allocated_votes_party_C_pt = votes_party_C_p * allocation_weight_pt
```

Repeat for all parties.

This assumes that the party composition of the poll is distributed across its internal population according to the same population weights.

Step 10: Aggregate allocated votes to census tracts

Group by census tract `ct_id`.

For each census tract `t`, calculate:

```
estimated_total_votes_t = sum(allocated_total_votes_pt)
```

For each party:

```
estimated_votes_party_A_t = sum(allocated_votes_party_A_pt)
estimated_votes_party_B_t = sum(allocated_votes_party_B_pt)
estimated_votes_party_C_t = sum(allocated_votes_party_C_pt)
...
```

The result is the estimated election result for each census tract.

Step 11: Calculate party shares

For each census tract `t`, calculate party vote shares.

For party A:

```
estimated_party_share_A_t = estimated_votes_party_A_t / estimated_total_votes_t
```

For party B:

```
estimated_party_share_B_t = estimated_votes_party_B_t / estimated_total_votes_t
```

For party C:

```
estimated_party_share_C_t = estimated_votes_party_C_t / estimated_total_votes_t
```

Repeat for all parties.

If:

```
estimated_total_votes_t = 0
```

then party shares should be set to null or missing.

Step 12: Validate that vote totals are preserved

The method should preserve the original poll-level vote totals.

Check total votes:

```
sum(estimated_total_votes_t across all census tracts)
≈
sum(total_votes_p across all polls)
```

Check party votes:

```
sum(estimated_votes_party_A_t across all census tracts)
≈
sum(votes_party_A_p across all polls)
```

```
sum(estimated_votes_party_B_t across all census tracts)
≈
sum(votes_party_B_p across all polls)
```

```
sum(estimated_votes_party_C_t across all census tracts)
≈
sum(votes_party_C_p across all polls)
```

Small floating-point differences are acceptable.

Large differences indicate a problem with geometry, clipping, missing overlaps, or normalization.

Step 13: Add useful quality flags

For each census tract, calculate fields that help describe uncertainty.

Recommended fields:

```
num_source_polls_t
```

Number of polls that contributed votes to the census tract.

```
share_from_largest_poll_t
```

The share of the census tract's estimated votes that came from its largest contributing poll.

```
fallback_area_weight_used_t
```

Whether any part of the tract estimate used area weighting because population weights were unavailable.

```
estimated_total_votes_t
```

Very low estimated vote totals should be treated carefully.

Useful interpretation:

```
If share_from_largest_poll_t is high, the estimate is mostly based on one poll
and may be more stable.

If num_source_polls_t is high, the tract estimate is assembled from many polls
and may involve more interpolation uncertainty.

If fallback_area_weight_used_t is true, the estimate should be treated with more
caution.
```

Step 14: Optional approximate turnout estimate

If the census tract has a population denominator, approximate turnout can be calculated.

For example, if using citizens above 18:

```
estimated_turnout_t = estimated_total_votes_t / citizens_above_18_t
```

If using voting-age population:

```
estimated_turnout_t = estimated_total_votes_t / voting_age_population_t
```

This should be labelled as approximate because the numerator is interpolated.

Summary of the core formula

For every poll `p` and census tract `t`:

```
pop_weight_pt = estimated relevant population inside p ∩ t
```

Normalize within each poll:

```
allocation_weight_pt = pop_weight_pt / sum(pop_weight_pt for all t overlapping
p)
```

Allocate total votes:

```
allocated_total_votes_pt = total_votes_p * allocation_weight_pt
```

Allocate party votes:

```
allocated_votes_party_k_pt = votes_party_k_p * allocation_weight_pt
```

Aggregate to census tracts:

```
estimated_votes_party_k_t = sum(allocated_votes_party_k_pt for all p overlapping
t)
```

Calculate party share:

```
estimated_party_share_k_t = estimated_votes_party_k_t / estimated_total_votes_t
```

Plain-language description

This method estimates census-tract-level election results by reallocating poll-level votes according to the distribution of population within each poll. First, we intersect election polls with census tracts. Then we use a smaller population geography to estimate how much relevant population lies inside each poll-tract overlap. For each poll, these population estimates are normalized so that all overlapping census tracts receive a share of the poll's votes adding up to 100%. We then allocate total votes and party-level votes from each poll to census tracts using those normalized weights. This preserves the official poll-level vote totals while producing census-tract estimates that better reflect where people live than a simple area-weighted method.